

AI-Native SDLC 报告附录

本附录是主报告的自包含补充。它做三件事：提供完整的证据索引，固定术语口径，提供落地检查清单。

1. 证据索引

本章按主报告章节组织，列出支撑各章核心判断的关键事实。每条证据给出来源名称、关键事实摘录、注意事项和原始 URL。读者在主报告中看到"（详见附录 证据 EXX）"时，可在此找到完整描述。

第 1 章：为什么不是"更快写代码"

证据 E01：Salesforce — AI 后代码量增 30%，瓶颈转移到 safe shipment

来源：Salesforce Engineering, *How AI-Enabled Tooling Boosted Code Output 30% While Keeping Quality and Deployment Safety Intact*

关键事实：

- Salesforce 报告 AI 工具规模化部署后，进入生产的代码量增加了 30%
- 文章明确说瓶颈从"代码创建"转移到了"让代码安全上线"（preparing that code for safe shipment）
- 团队的应对不是"更信任 AI"，而是加大对验证 workflow、测试覆盖自动化和 review 支持的投资
- Salesforce 明确规定：生成式或推理型 AI 不允许自主变更生产系统——人类仍然是生产影响决策的最终责任人

注意事项：这是一家成熟大型企业的工程实践报告，有内部平台能力支撑。小型团队的瓶颈转移路径可能不同，但方向一致。

原始

URL：

<https://engineering.salesforce.com/how-ai-enabled-tooling-boosted-code-output-30-while-keeping-quality-and-deployment-safety-intact/>

证据 E02：Salesforce — Review 在 AI 负载下的崩塌（相关证据，与 E01 同源公司不同文章）

参见 E03。E01 和 E02 共同描述了 Salesforce 的两面：一面是代码量增加，另一面是 review 崩塌。

证据 E03：Salesforce — Code Review 的"第二双眼睛保证"侵蚀

来源：Salesforce Engineering, *Scaling Code Reviews: Adapting to a Surge in AI-Generated Code*

关键事实：

- AI 辅助开发增加了代码体量，PR 大小扩展到人工有效审查范围之外
- Review 延迟增加——即使写代码的时间缩短了
- 文章明确说更深层的问题是"第二双眼睛保证"（second-pair-of-eyes guarantee）的侵蚀
- 可预测的失败模式包括：大 PR 导致概念连贯性丧失、reviewer 认知负荷非线性增长、reviewer 参与度下降

注意事项：这是一家公司的工程报告，不是对所有组织的通用证明。但其描述的失败模式具有结构性——只要代码产出速度超过 review 消化速度，类似的崩塌就会出现。

原始 URL：<https://engineering.salesforce.com/scaling-code-reviews-adapting-to-a-surge-in-ai-generated-code/>

证据 E04：ThoughtWorks — 闭门研讨会发现与战略洞察

来源：ThoughtWorks, *The Future of Software Engineering: Retreat Findings and Strategic Insights*, 2026 年 2 月

关键事实（涉及主报告多个章节，此处综合列出）：

关于瓶颈转移（Ch1）：

- 从业者报告：给团队 AI 工具后，他们在几天内清空了积压工作，然后撞上了跨团队依赖、架构评审和人类速度决策的墙壁
- 结果不是更快的交付，而是同样的速度加上更多的挫败感

关于 TDD（Ch3）：

- 从业者原话："TDD 配合 Agent 编码获得了前所未有的最好结果——因为它阻止了一种特定的错误：Agent 编写测试来验证自己的错误行为"
- 研讨会将 TDD 重新定义为"一种提示工程"——测试成为非确定性生成的确定性验证

关于大批量倒退（Ch3）：

- 研讨会明确标记"用 AI 工具轻松产出大型变更集正在推动团队回到类似瀑布的模式"为"一个正在发生的倒退"
- 这是对 DORA 十年研究成果（小批量 = 高稳定性）的直接逆转

关于 Agent 规避规则（Ch5）：

- 真实案例：一个有 linter 访问权限的 Agent，面对 500 行文件限制规则，选择把单行代码变长来规避——技术上满足了规则但违反了规则背后的原则
- 多个 Agent 同时修复同一问题时产生反馈循环，一个的修复触发另一个的纠正，系统振荡而非收敛

关于电信公司知识图谱（Ch5）：

- 一家大型电信公司的整个领域本体可以用约 286 个概念来捕获——这个数字让工作变得"可实现"
- 团队用 LLM 自动从代码中识别命令、事件、聚合和策略——自动生成事件风暴制品——再由人类专家验证和纠正
- 把数周的发现研讨会压缩到几天

关于安全（Ch6）：

- 安全讨论环节出席率低，反映了行业模式——安全被当成"等技术跑通了再说"
- 最生动的攻击场景：授予 Agent 电子邮件访问权限 → 密码重置 → 账户接管
- 研讨会建议：平台工程应该让安全行为简单、不安全行为困难

关于角色变化（Ch4）：

- 中间循环给热爱编程的开发者制造了真正的身份认同危机——许多人最初恰恰是为了"把工单转化为可工作的代码"被雇用，而这项工作正在消失
- Staff 工程师比以前更重要也更有压力：使用 AI 频率低于初级工程师，但使用时每周节省的时间更多
- 初级开发者比以往更有价值——AI 帮他们更快度过净负值阶段，而且他们更擅长使用 AI 工具

注意事项：研讨会遵循查塔姆宫规则，不披露参与者姓名或所属机构。内容是定性观察和经验汇聚，不是量化研究。但参与者是来自大型科技公司的资深工程从业者，观察具有行业代表性。

原始

URL：

https://www.thoughtworks.com/content/dam/thoughtworks/documents/report/tw_future_of_software_development_retreat_key_takeaways.pdf

证据 E05：DORA — AI 使用与交付稳定性的反直觉关系

来源：DORA, *Impact of Generative AI in Software Development*

关键事实：

- 随 AI adoption 增加，文档质量、代码质量、review 速度和审批速度都有正向变化
- 但同一报告也给出负面结果：交付吞吐量（delivery throughput）可能轻微下降，交付稳定性（delivery stability）下降更明显
- 这意味着"AI 让个体更快"与"团队交付更稳"不是同一个命题
- 报告特别适合校正"代码更多 = 软件交付更好"这种错误推断

注意事项：该报告讨论的是 adoption 与 outcome 的统计关联，不等于对所有组织给出直接因果证明。但方向性信号足够清晰。

原始 URL： <https://dora.dev/research/ai/gen-ai-report/dora-impact-of-generative-ai-in-software-development.pdf>

证据 E06：DORA 2025 — AI 是组织的放大器

来源：DORA 2025 Report

关键事实：

- DORA 说 AI 的主要作用是放大组织原有的优势和弱点
- 最大的 AI 投资回报来自改善底层组织系统本身，而不仅仅是部署工具
- 这意味着治理延迟、团队拓扑和决策流都是 AI-Native 软件交付中的一阶关注

注意事项：这是报告摘要页面，确立了上层命题但未详述每种组织失败模式的具体机制。

原始 URL： <https://dora.dev/research/2025/dora-report/>

第 3 章：工程纪律的上移

证据 E07：WebApp1K — Tests-as-Prompt 的基准验证

来源： *Tests as Prompt: A TDD Benchmark for AI Code Generation* (arXiv)

关键事实：

- 论文引入 WebApp1K benchmark，其中测试用例同时作为 prompt 和验证信号
- 论文认为这种设置比单纯依赖自然语言 prompt 更接近真实开发场景
- 结果强调：instruction following 和 in-context learning 对 TDD 成功率的影响比通用编码能力更大
- 也暴露了瓶颈：长 prompt 中的指令丢失和多特性复杂度

注意事项：Benchmark 证据有助于理解机制，但不等同于企业部署证据。需要与结构化需求和更小的任务分解结合使用。

原始 URL： <https://arxiv.org/abs/2505.09027>

证据 E08：LLM4TDD — AI TDD 的监督需求

来源： *LLM4TDD: Best Practices for Test Driven Development Using Large Language Models* (学术论文)

关键事实：

- 研究证实：没有增量约束的 AI TDD 会生成迎合提示的代码——Agent 首先生成代码，然后写一个"验证自己正确"的测试
- 需要人类监督或结构化的约束注入来打破这种循环

注意事项：学术实验环境，不是企业部署数据。但暴露的失败模式具有通用性。

原始 URL： <https://arxiv.org/abs/2312.04687>

证据 E09：LLM4TDG — 约束推理提升 Agent 修复能力

来源： *Constraint Reasoning for Test Data Generation with LLMs* (学术论文)

关键事实：

- 研究表明显式约束能提高 Agent 的修复能力和正确性
- 当约束以依赖图形式呈现时，Agent 更能理解修复的连锁影响

原始 URL： <https://link.springer.com/article/10.1186/s42400-024-00335-4>

证据 E10: DORA — 小批量交付的十年证据

来源: DORA, *Working in Small Batches* (capability guide)

关键事实:

- DORA 明确说较小的批量与更高的交付稳定性相关——这是十年研究的核心结论之一
- 小批量的定义不仅是"代码行数少", 而是"未验证意图的暴露范围小"
- 小批量需要配套的工程基建: trunk-based development、短生命周期分支、快速 CI、合并保护

注意事项: 这是 DORA capability 指南, 不是单一研究论文, 综合了多年的数据。

原始 URL: <https://dora.dev/capabilities/working-in-small-batches/>

第 4 章: 人类控制面

证据 E11: GitHub — AI Champions 网络

来源: GitHub, *Activating Internal AI Champions*

关键事实:

- GitHub 说 AI 采用不只是技术问题, 而是变革管理问题
- 内部 advocate 不靠行政权力运作, 而是通过 peer-to-peer 的信任
- 他们的功能包括: 让 AI 变得实用、帮助同事越过早期障碍、作为从团队到领导层的反馈回路

注意事项: 这是 GitHub 的 playbook 文章, 是处方性和经验性的, 不是独立的比较研究。

原始 URL: <https://github.com/resources/insights/activating-internal-ai-champions>

证据 E12: Thomson Reuters — 分阶段 AI 采用

来源: GitHub, *Thomson Reuters AI Adoption Case Study*

关键事实:

- Thomson Reuters 的 AI 推广包括分阶段 rollout、配套培训、指标驱动的采用跟踪和正式的 champion program
- 这是目前公开的、最完整的企业级 AI 采用的具体操作模型之一

注意事项: 由 GitHub 撰写的案例研究, 侧重采用流程而非安全或治理维度。

原始 URL: <https://github.com/resources/customer-stories/thomson-reuters>

证据 E13: Microsoft Work Trend Index 2025 — AI 人力资源管理者

来源: Microsoft, *Work Trend Index 2025: The Year the Frontier Firm Is Born*

关键事实:

- Microsoft 调查覆盖多家企业, 说每个员工将越来越多地成为"agent boss"
- 28% 的管理者正在考虑招聘 AI 人力资源管理者来领导人类-Agent 混合团队
- 32% 计划在未来 12-18 个月内招聘 AI Agent 专家来设计、开发和优化 Agent
- 领导者预期更多工作将围绕用 AI 重新设计业务流程、构建多 Agent 系统、训练和管理 Agent

注意事项: 这是广泛工作场所调查, 不仅限于软件工程。招聘数字反映的是计划和期望, 不是已确立的新角色。

原始 URL: <https://www.microsoft.com/en-us/worklab/work-trend-index/2025-the-year-the-frontier-firm-is-born>

证据 E13b: Atlassian Teamwork Graph — 共享上下文层

来源: Atlassian, *What is Teamwork Graph*

关键事实:

- Atlassian 描述 Teamwork Graph 为跨 Jira、Confluence、Google Drive、Slack、GitHub 等系统的公共数据模型
- 连接器 (connectors) 把第三方和内部数据映射为共享对象类型、关系和权限
- 一旦数据进入 Teamwork Graph, 就可以跨 Rovo Search、Chat、Agents 和 Analytics 使用
- Atlassian 明确将这个图与跨工具自动化、统一搜索、推荐和工作流分析关联

注意事项: 这是一个平台能力描述, 不是采用结果的现场研究。它展示了工具面可以是什么样, 但不证明团队是否成功围绕它重组。

原始 URL: <https://developer.atlassian.com/platform/teamwork-graph/what-is-teamwork-graph/>

第 5 章: Agent Runtime

证据 E14: Anthropic — 多智能体研究系统

来源: Anthropic Engineering, *Building a Multi-Agent Research System*

关键事实:

- Anthropic 描述了 lead-agent + subagent 的 orchestrator-worker 模式
- 他们明确说: 多 Agent 系统引入了新的协调、评估和可靠性挑战
- Lead agent 把计划存在 memory 中, 这样即使上下文被重置, 计划也可以存续并在子 agent 之间传递
- Subagent 使用外部 artifact creation, 把产出直接持久化而不是通过对话历史传递
- 系统依赖 durable execution 和 resume capability——从头重跑长时 agent 的成本太高
- **Anthropic 也坦承: 多 Agent 系统很贵, 而且对紧密耦合的任务不太适用**

注意事项: 这是 Anthropic 自己的生产研究系统, 不是通用企业架构。但它提供了目前公开最具体的多 Agent 工程决策描述。

原始 URL: <https://www.anthropic.com/engineering/multi-agent-research-system>

证据 E14b: Anthropic — Context Engineering for Agents

来源: Anthropic, *Effective Context Engineering for AI Agents*

关键事实:

- Anthropic 描述了 Agent 上下文管理的核心挑战: note-taking、compaction 和 sub-agent 架构
- 支撑 Agent OS / 工作账本概念中的上下文管理设计

原始 URL: <https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/context-engineering-agents>

证据 E14c: Temporal — 持久执行的事件历史

来源: Temporal Documentation, *Event History*

关键事实:

- Temporal 描述 Event History 为工作流执行生命周期的完整、持久日志
- 工作流代码不直接执行外部操作——它发出命令, Temporal 服务将其映射为持久化事件
- Worker 崩溃后, 工作流从事件历史重放以重建执行状态并从失败点恢复
- 平台把 resumability 视为**服务端持久性属性**, 不是应用层的事后补丁

注意事项：Temporal 是通用持久 workflow 系统，不是 agent-native 账本产品。它覆盖执行历史和重放，但不覆盖 agent 权限、预算、技能和验收标准的统一模型。

原始 URL： <https://docs.temporal.io/encyclopedia/event-history>

证据 E14d: MemGPT — 操作系统式的记忆管理

来源： *MemGPT: Towards LLMs as Operating Systems* (arXiv)

关键事实：

- MemGPT 提出"虚拟上下文管理" (virtual context management)，灵感来自操作系统的分层记忆系统
- 论文说有限的 context window 是扩展对话和文档分析的硬约束
- 引入多层记忆和类中断 (interrupt-like) 控制流来管理交互
- 框架明确是 OS 式的，而不是纯粹的 prompting

注意事项：这是学术研究，不是标准企业基础设施。在记忆架构上很强，但在协议互操作性和可观测性上较弱。

原始 URL： <https://arxiv.org/abs/2310.08560>

第 6 章：安全与发布

证据 E15: CircleCI — 2023 年安全事故

来源： CircleCI, *January 4, 2023 Security Incident Report*

关键事实：

- 攻击者在工程师笔记本上植入恶意软件，窃取了有效的 2FA SSO 会话，然后提权进入一个生产系统子集
- 攻击者从运行中的进程提取了加密密钥，使得"静态加密"对已泄露数据的保护大打折扣
- 客户存储在 CircleCI 中的环境变量、tokens 和第三方系统密钥被泄露
- CircleCI 要求所有客户轮换密钥，并在事后扩展了审计日志访问和密钥检查工具
- CircleCI 计划的修复包括：step-up 认证、更频繁的 token 轮换、向 GitHub Apps 迁移以获得更细粒度权限、更短时效的内部权限

注意事项：这是 CI 平台事故，不是 Agent 特有事故。但它最清楚地说明了为什么 CI/CD 系统是高爆炸半径的控制面——它同时持有代码访问、凭证存储、制品生成和部署权限。当 Agent 连接 CI/CD 工具时，同样的风险面被继承。

原始 URL： <https://circleci.com/blog/jan-4-2023-incident-report/>

证据 E16: GitHub Actions — 安全使用指南

来源： GitHub, *Security Guides for GitHub Actions*

关键事实：

- GitHub 说将 action 固定到 full-length commit SHA 是目前使用不可变 release 的唯一方式
- GitHub 推荐 OpenID Connect 进行云访问——让 workflow 使用短期、限定范围的 token 而不是长期 secrets
- GitHub 托管的 runner 在短暂的干净虚拟机中运行；自托管 runner 没有同样的保证
- GitHub 文档化了 just-in-time runners——最多执行一个 job 就自动删除
- GitHub 提供了 Actions 事件的审计日志和依赖审查功能

原始 URL： <https://docs.github.com/en/actions/reference/security/secure-use>

证据 E17: GitHub — 分支保护与部署闸门

来源: GitHub, *About Protected Branches*

关键事实:

- GitHub 保护分支可以要求 PR reviews、status checks、merge queue 和部署成功才允许合并
- Merge queue 确保 PR 变更在应用到最新目标分支和所有已在队列中的 PR 时都能通过 required checks
- GitHub 文档化了一条规则: 可以要求变更**成功部署到特定环境后才能合并到分支**
- 分支限制使得只有特定用户、团队或 apps 能推送到保护分支

原始

URL:

<https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches>

证据 E18: Google Binary Authorization — 部署时策略强制

来源: Google Cloud, *Binary Authorization Overview*

关键事实:

- Google 说 Binary Authorization 可以通过持续验证监控策略一致性, 对受支持的容器平台执行部署策略
- Attestation 可以验证镜像是否由特定构建系统或 CI 管道构建
- 策略可以在部署前要求 attestation——必需的签名者必须在镜像进入下一部署阶段前创建 attestation
- Deploy-time enforcer 阻止违反策略的镜像, 并将解释写入 Cloud Audit Logs
- Google 还文档化了部署后的持续验证 (continuous validation)

原始 URL: <https://docs.cloud.google.com/binary-authorization/docs/overview>

证据 E19: Google WIF — 联邦身份审计链

来源: Google Cloud, *Best Practices for Using Workload Identity Federation*

关键事实:

- Google 明确推荐为 Security Token Service API 和 IAM API 启用数据访问日志, 使 impersonation 事件可审计
- 文档说 serviceAccountDelegationInfo 部分可以帮助识别 impersonate 了服务账户的主体
- Google 强调 google.subject 映射必须双向唯一——这样外部身份可以从审计日志可靠地追溯回来
- 该指南还强调最小权限——限制哪些外部身份可以 impersonate 服务账户, 以及每个服务账户可以访问什么

注意事项: 聚焦 Google Cloud 联邦身份, 不是云中中性架构。在审计和属性映射上很强, 在事故案例研究上较弱。

原始 URL: <https://docs.cloud.google.com/iam/docs/best-practices-for-using-workload-identity-federation>

证据 E20: OWASP — Agentic AI Top 10 (2026)

来源: OWASP GenAI Security Project, *Top 10 for Agentic Applications 2026*

关键事实:

- OWASP 将其描述为一个全球 peer-reviewed 框架, 覆盖自主和 agentic AI 系统面临的最关键风险
- 框架旨在为构建者、防御者和决策者提供保护 Agent 的实用起点——这些 Agent 跨 workflow 进行规划、行动和决策
- 识别的风险包括: Agent Goal Hijack、Tool Misuse、Identity & Privilege Abuse、Agentic Supply Chain Vulnerabilities、Unexpected Code Execution

注意事项: 社区驱动的指南, 相比 NIST ZTA 等成熟框架仍然较新。建议与 NIST SP 800-207 配合使用。

原始 URL: <https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/>

证据 E21：NIST SP 800-207 — 零信任架构

来源：NIST, *Zero Trust Architecture*

关键事实：

- ZTA 将保护重点从网络边界转移到资源、身份与工作流
- 核心原则：从不隐式信任、始终验证、最小权限、假设已被攻破、基于身份而非边界、持续验证

原始 URL： <https://csrc.nist.gov/pubs/sp/800/207/final>

第 7-8 章相关

证据 E22：DORA — 清晰的 AI 使用策略

来源：DORA Capability Guide, *Clear AI Stance*

关键事实：

- DORA 说有清晰 AI 使用策略的组织，其采用摩擦更小，并且更可能获得正向结果
- 关键不是策略有多严格，而是**团队是否真正理解并遵守**

原始 URL： <https://dora.dev/capabilities/clear-ai-stance/>

证据 E23：DORA — 信任校准

来源：DORA Research Insight, *Trust in AI*

关键事实：

- DORA 说对 AI 的信任水平直接影响生产力表现
- 关键不是"信不信 AI"——而是团队是否有结构化的方式来**校准**对不同任务的信任程度：哪些任务可以把 AI 输出直接合并，哪些需要人工审查

原始 URL： <https://dora.dev/research/ai/trust-in-ai/>

证据 E24：GitHub — Executive Support Playbook

来源：GitHub, *Executive Support Playbook for AI Adoption*

关键事实：

- GitHub 明确要求 executive-level 翻译和 manager enablement 作为 AI 采用的结构性组件
- 管理者需要能够讨论 AI proficiency——不只是关心产出数字

原始 URL： <https://github.com/resources/insights/executive-support-ai-adoption>

2. 术语表

控制栈 Control Stack

一句话解释：把企业在 AI-Native SDLC 中所有需要"管住"的能力——从安全身份到质量闸门到组织协调——叠成一个从下到上的四层结构来看。

正式定义：由基底层（安全与治理）、运行层（Agent Runtime）、执行层（工程纪律）和协调层（人类控制面）四层组成的控制体系。每层依赖下面的层才能正常工作。

不等于：只是"加几个安全检查"或"改一下 CI/CD 配置"。

中间循环 Middle Loop

一句话解释：介于"写代码"和"交付产品"之间的人类控制工作——谁来拆任务、谁来校准 Agent 的输出、谁来翻译策略、谁来决定什么时候升级处理。

正式定义：位于代码生成与最终交付之间的人类控制面工作，覆盖任务拆解（routing/decomposition）、上下文打包（context packaging）、信任校准（trust calibration）、策略翻译（policy translation）、升级决策（escalation）和采用反馈（adoption feedback）。

不等于：某一个固定岗位名称。它是一组能力，可能分布在 Champion、Manager、PM、Staff Engineer、Platform Engineer 等多个角色身上。

智能体体验 AgentEx (Agent Experience)

一句话解释：Agent 工作流的运行质量——Agent 是否可控、上下文是否充分、运行时是否稳定、交接是否顺畅。

正式定义：Agent workflow 的运行质量指标，包括可控性（controllability）、上下文质量（context quality）、运行时人机工程学（runtime ergonomics）、可靠性（reliability）和交接质量（handoff quality）。

与 DevEx 的关系：AgentEx 不替代 DevEx，而是与之并存。成熟的系统需要同时优化人类体验和 Agent 工作流体验。这一术语来自 ThoughtWorks 研讨会从业者的提议："停止称之为开发者体验，改称智能体体验——这样钱包会更快打开。"（详见附录 证据 E04）

智能体操作系统 Agent OS

一句话解释：Agent 稳定运行需要的系统软件层——编排、记忆、协议、工件持久化、可观测性，类似于 Kubernetes 之于容器。

正式定义：支撑 Agent 稳定运行的分层运行时系统，至少包含 orchestration（编排）、external memory（外部记忆）、tool/agent protocols（工具/智能体协议）、artifact persistence（工件持久化）、observability（可观测性）和 policy hooks（策略挂钩点）。

不等于：单一模型服务端点、IDE 插件，或某个供应商的产品名称。当前更像架构模式（architecture pattern），而不是成熟产品类别。

工作账本 Work Ledger

一句话解释：面向 Agent 任务的"系统记录簿"——把任务状态、权限、预算、工具使用、验收标准和执行历史挂到一个统一的控制面上。

正式定义：面向 Agent 任务的复合记录系统，用来承载 run identity（执行标识）、execution history（执行历史）、checkpoints（检查点）、artifacts（工件）、ownership boundary（所有权边界）、traces（追踪），以及未来可能扩展的 capability（能力）、budget（预算）和 acceptance criteria（验收标准）。

当前状态：战略上重要，底层 primitives 已经存在（Temporal、LangGraph、OpenAI Agents SDK、Inngest），但公开的统一企业级设计尚未出现。（详见附录 证据 E14c）

爆炸半径 Blast Radius

一句话解释：一个 Agent 操作如果出错或被滥用，它的下游技术和业务影响能波及多大范围。

正式定义：一次 Agent action、tool call、workflow change 或 generated change 如果出错或被滥用，其下游技术与业务影响范围。评估维度包括：代码变更的跨模块影响、凭证暴露范围、环境访问权限、工件完整性、供应链依赖等。

当前状态：环境级和身份级的评估模型越来越清晰（graph-based exposure scores、attack path analysis）；diff 级和 tool-call 级的评分仍不成熟。CircleCI 事故（证据 E15）是理解 CI/CD 平台作为高爆炸半径控制面的最佳案例。

小批量执行 Small-Batch Enforcement

一句话解释：一组平台层面的控制机制，防止 AI 生成的变更积累成大批量、低频、难以 review 的发布。

正式定义：让 AI-generated changes 保持短生命周期（short-lived branch）、易验证（fast CI）、易回滚（canary exposure）的控制链，包括 work slicing、protected merge、merge queue、deployment gate 等。

重点：不是"PR 行数限制"，而是"未验证意图和过大生产暴露比例的系统性抑制"。（详见附录 证据 E10）

发布风险闸门 Release-Risk Gate

一句话解释：把工程质量闸门和安全闸门合并成一个统一的发布准入系统。

正式定义：将 engineering gate 和 security gate 合并的统一发布准入系统，至少覆盖：不可变 workflow 依赖（immutable workflow dependencies）、短期凭证（short-lived credentials）、必需检查（required checks）、保护合并路径（protected merge path）、部署成功验证（deployment success gate）、制品溯源（artifact provenance）、环境保护（protected environments）和审计追溯（audit trail）。

不等于：一个万能风险评分。它是一组可独立配置的控制组件。（详见附录 证据 E16, E17, E18）

零信任 Zero Trust Architecture (ZTA)

一句话解释：不因为"这个请求来自内网"就自动信任它——每次访问都要验证身份、检查权限、记录审计。这恰好也是 Agent 安全的最佳描述语言。

正式定义（根据 NIST SP 800-207）：一种安全模型，将保护重点从网络边界转移到资源、身份与工作流。核心原则包括：从不隐式信任、始终验证、最小权限、假设已被攻破、基于身份而非边界、持续验证。

对 Agent 的特殊意义：Agent 本质上就是新的 non-human actor，ZTA 天然适合描述其身份管理、工具授权和运行时监控。（详见附录 证据 E21）

3. 落地检查清单

这份清单对应主报告第 7 章的落地顺序。用于团队自评当前在 AI-Native SDLC 控制栈上的准备度：

Phase 0 检查：基线可见性

- ☐ 是否已统一关键技术语定义？（团队是否在同一语言体系下讨论？）
- ☐ 是否已建立 DORA 类交付指标的基线？（部署频率、交付前置时间、变更失败率、故障恢复时间）
- ☐ 是否已有明确的、被团队理解的 AI 使用策略（AI Stance）？（不是法务文件，而是团队真正理解的行为准则）（详见附录 证据 E22）

Phase 1 检查：护栏先于自治

- ☐ 规格产物是否已升级为可被 Agent 和 CI 同时消费的结构化对象？
- ☐ 测试套件是否已前置为任务定义和意图约束的一部分？
- ☐ 是否已启用 required checks + merge queue + deployment gate？（详见附录 证据 E17）
- ☐ 是否已建立短期凭证（short-lived non-human identity）机制？（详见附录 证据 E19）
- ☐ 是否已对高等级环境启用 protected environments / deployment approval？

Phase 2 检查：人类控制面

- ☐ 是否已有 Champion 网络在做示范和反馈收集？（详见附录 证据 E11, E12）
- ☐ Manager 是否能讨论 AI proficiency（而不只是关心产出数字）？（详见附录 证据 E24）
- ☐ 是否有策略翻译机制——把组织的安全和质量策略转化为 Agent 能理解的约束？
- ☐ 是否有采用指标和反馈回路——知道哪些 AI 使用方式有效、哪些无效？
- ☐ 是否有升级路径——当 Agent 输出超出边界时的处理流程是否定义清楚？

Phase 3 检查：Agent Runtime

- ☐ 工具连接是否已标准化（MCP 或等效协议）？
- ☐ 长流程 Agent 是否有 checkpoint 和 artifact 持久化？（详见附录 证据 E14, E14c）
- ☐ 是否有统一的 tracing 和可观测性（OTel 或等效方案）？
- ☐ Agent 中断时是否有定义好的恢复机制？

Phase 4 检查：受控扩展

- ☐ 多 Agent 场景是否经过显式的任务结构评估？（不是因为"酷"才上多 Agent）
- ☐ 多 Agent 工作流是否要求 resumability、telemetry 和 policy hooks？

Phase 5 检查：持续治理

- ☐ 红队测试是否从一次性活动变成了持续能力？
- ☐ 是否有 posture drift 检测机制？
- ☐ 是否显式记录了哪些 AI 能力仍然标记为 not yet mature？

4. 本报告的判断类型说明

主报告中使用了五种内容类型标签，帮助读者区分不同信心水平的内容：

标签	含义	如何使用
硬事实	有直接、多源证据支持的客观事实	可以直接拿来做设计决策的依据
分析判断	基于证据链推导出的结构化判断	可以参考，但需要结合自身场景验证
实施建议	基于当前最佳实践的操作建议	可以作为起点，但需要根据企业具体情况调整
趋势推测	基于方向性信号的演进预测	作为规划参考，不应作为硬性决策依据
开放问题	高价值但尚未收敛的问题	列入研究和观察 roadmap，不要伪装成已解决